ONDC Protocol RAG: Baseline Retrieval vs. Re-Ranking

Language Models and Agents
Paila Tejeswara Rao

Abstract. This project implements a retrieval-augmented generation (RAG) checker for the ONDC eB2B API contract. It targets a practical product-development question: can a proposed buyer-app or seller-app feature be supported by the protocol? Two pipelines are compared on the same frozen corpus and prompt: dense retrieval using a bi-encoder, and dense retrieval followed by cross-encoder re-ranking. The evaluation focuses on whether relevant but initially buried protocol passages reach the final context, whether the generated answer becomes more complete, and what latency is added by re-ranking.

1. Problem and Objective

During product development, teams working on retailer buyer apps and distributor seller apps need to verify proposed features against ONDC protocol rules before implementation or release. Manually searching a long, schema-heavy contract is slow, and ordinary wording used by a product team may differ from the protocol's terminology. The objective is to let a user ask a question in natural language and receive an answer grounded only in retrieved protocol passages.

The source selected was the *ONDC eB2B API Contract v1.2.5* (99 PDF pages, approximately 2.4 MB and 259K extracted characters). Text was available on 97 pages. Two smaller documents were rejected because their 13–15 chunks were too few for meaningful Top-10 retrieval and rank-shift analysis. The chosen contract contains buyer/seller network participant flows, JSON schemas, error codes, settlement fields, and fee-related rules, matching the target use case.

2. Implementation Pipeline

2.1 1. PDF ingestion and chunking

The PDF is read page by page using `pypdf`; empty extracted pages are skipped. Each non-empty page is split into character windows of 1,000 characters with 150 characters of overlap. Chunks shorter than 200 characters are discarded. Every chunk stores its text, source page, and an identifier (for example, `p82_283`). This produced 328 chunks. Chunking is run once and the same corpus is used for both pipelines, so the comparison does not confound retrieval strategy with different input data.

2.2 2. Embedding and dense retrieval (Pipeline 1)

The `all-MiniLM-L6-v2` bi-encoder maps each chunk to a 384-dimensional vector. Chunk vectors are L2-normalized and stored in a FAISS `IndexFlatIP` index. For normalized vectors, inner product is equivalent to cosine similarity:

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \, \|d\|}.$$

At query time, the question is encoded and normalized in the same way. FAISS returns the Top 10 chunks with their similarity scores, identifiers, and page numbers. The baseline pipeline takes the first three results directly as the generation context.

2.3 3. Cross-encoder re-ranking (Pipeline 2)

The same Top 10 results are passed to `cross-encoder/ms-marco-MiniLM-L-6-v2`. Unlike the bi-encoder, which encodes query and document independently, the cross-encoder scores each (query, chunk) pair jointly. The ten scores are sorted in descending order and the new Top 3 are sent to the language model. Re-ranking is restricted to these ten candidates: this retains fast corpus-wide retrieval while allowing a more context-aware comparison of the most promising passages.

2.4 4. Grounded answer generation

Both pipelines use the same prompt template and the same Gemini model within each comparison. The prompt instructs the model to answer only from the three supplied chunks and to say "not found" when the evidence is absent. The notebook uses `models/gemini-3.8-flash`, with `gemini-flash-lite-latest` as a fallback after quota limits. The API key is loaded from Colab Secrets rather than hard-coded. Consequently, the intended experimental difference between Answer 1 and Answer 2 is the selected context, not the prompt or corpus.

PDF → page text → overlapping chunks → embeddings + FAISS Top 10
↙ Top 3 directly Top 10 pairs → Cross-Encoder → new Top 3
Same strict prompt + Gemini → Answer 1 (baseline) / Answer 2 (re-ranked)

3. Evaluation and Findings

Evaluation queries were designed to test keyword mismatch, exception handling, negation/constraints, and protocol details buried below the initial Top 3. Rank shifts alone were treated as supporting evidence; a stronger scoring case also required an answer-quality difference or a meaningful improvement in grounded detail.

3.1 Primary scoring cases

- **Keyword mismatch – miscellaneous fees.** The natural-language query asked whether a buyer app could add convenience fees. The protocol wording referred to miscellaneous fees at fulfillment level and stated that the BNP customizes the displayed title. The baseline retrieved a related chunk but produced an incomplete answer, saying the title-text detail was not found. Re-ranking promoted chunk `p82_283` from rank 4 to rank 1, and Answer 2 included both the fee context and BNP title customization.
- **New-retailer exception.** The question asked whether a new retailer can order before distributor mapping is complete. The general mapping rule and the `type=new` exception must be interpreted together. The baseline answer was a short yes with limited explanation. Re-ranking promoted `p3_9` from rank 4 to rank 2, allowing Answer 2 to provide more protocol-specific detail, including distributor selection by the Seller NP, pending creation, and P1 naming. This is a precision/detail improvement rather than a complete fail-to-pass case, since both answers returned the broad yes.

3.2 Supporting rank-shift evidence

Additional tests showed that re-ranking can promote relevant details that dense retrieval initially placed lower:

Protocol detail	Old rank	New rank
Error codes for declined/lapsed retailer creation	10, 9	2, 3
NACK code 30000 when provider ID is omitted	6, 7	1, 3
Distributor must-buy rule	2	1
Feature codes 030 and 031	4, 6	1, 2
Retailer-mapping declarations	4, 5	1, 3

These cases demonstrate ranking changes, but not all are answer-quality wins: several already produced correct answers in the baseline. The evaluation therefore distinguishes retrieval improvements from actual generation improvements.

3.3 Latency and scaling

Retrieval timings were measured in Colab; retrieval-only values are averages over five runs, while end-to-end LLM timings were measured once per pipeline.

Operation	Measured time
Baseline FAISS Top-10 search	0.0075 s
Cross-encoder re-ranking of 10 pairs (additional)	0.0437 s
Pipeline 2 retrieval total (approx.)	0.051 s
Pipeline 1 end-to-end, including LLM	0.951 s
Pipeline 2 end-to-end, including LLM	0.732 s

Re-ranking costs about six times the baseline retrieval step, but adds only around 44 ms for ten candidates. End-to-end times are dominated by variable LLM latency; Pipeline 2 being faster in this single measurement should not be interpreted as evidence that re-ranking reduces generation latency. If the cross-encoder scored all 328 chunks instead of only ten, a rough linear estimate is about 1.4 seconds per query. At much larger corpus sizes, exhaustive cross-encoding becomes expensive, which motivates the retrieve-then-rerank design.

4. Discussion, Limitations, and Conclusion

The bi-encoder is efficient because chunk vectors can be precomputed and searched using FAISS, but it compares independently encoded representations and can miss paraphrases, constraints, or exceptions. The cross-encoder reads the query and each candidate together, often improving the order of relevant passages, but its per-pair computation is more expensive. A Top-10 prefilter followed by re-ranking balances these properties.

The strict grounded-answer prompt makes omissions visible and discourages unsupported claims, but it cannot compensate for missing evidence in the selected Top 3. Limitations were retained rather than hidden: a tax-display query changed from a partial baseline answer to "not found" after re-ranking; the original convenience-fee wording returned "not found" in both pipelines; and one declarations answer became terser after re-ranking. These outcomes show that promoting a relevant chunk does not guarantee a better final answer, and keyword-dense passages can narrow the model's response.

Overall, the implementation demonstrates a complete PDF-to-answer RAG pipeline and a controlled comparison of dense retrieval with cross-encoder re-ranking on an ONDC protocol document. The main result is that re-ranking can recover buried, terminology-mismatched details and improve answer completeness in selected cases, while adding a small and measurable retrieval cost. It remains a decision-support prototype: protocol-sensitive product decisions should be verified against the cited source passage and the applicable contract version.

Reproducibility. The notebook runs in Google Colab with the ONDC PDF uploaded to `/content`, Internet enabled, and the Gemini API key stored as `GEMINI_API_KEY` in Colab Secrets. Required packages are `pypdf`, `sentence-transformers`, `faiss-cpu`, and `google-generativeai`. Run the notebook cells in order; a runtime reset clears uploaded files and variables.